

# System wypożyczalni rowerów miejskich

Dokumentacja techniczna bazy danych

*System zarządzania bazą danych: PostgreSQL (≥ 12)*

Projekt z przedmiotu Bazy Danych

Mateusz Rybak, Maurycy Januszek

AGH Akademia Górniczo-Hutnicza im. Stanisława Staszica w Krakowie

Wydział Informatyki, Elektroniki i Telekomunikacji

Kraków, 2026

# Spis treści

|                                                                     |           |
|---------------------------------------------------------------------|-----------|
| <b>1. WPROWADZENIE I CEL PROJEKTU .....</b>                         | <b>4</b>  |
| <b>2. MODEL DANYCH, OPIS TABEL .....</b>                            | <b>5</b>  |
| 2.1. TARYFA, CENNIK USŁUG .....                                     | 5         |
| 2.2. KLIENT, UŻYTKOWNICY SYSTEMU .....                              | 5         |
| 2.3. STACJA, LOKALIZACJE WYPOŻYCZALNI .....                         | 6         |
| 2.4. ROWER, FLOTA POJAZDÓW .....                                    | 6         |
| 2.5. DOK, STANOWISKA DOKUJĄCE .....                                 | 6         |
| 2.6. WYPOŻYCZENIE, HISTORIA WYPOŻYCZEŃ .....                        | 7         |
| 2.7. PŁATNOŚĆ, ROZLICZENIA FINANSOWE .....                          | 7         |
| 2.8. PRACOWNIK, PERSONEL SERWISU .....                              | 8         |
| 2.9. SERWIS, ZGŁOSZENIA TECHNICZNE .....                            | 8         |
| <b>3. DIAGRAM ZWIĄZKÓW ENCJI (ERD).....</b>                         | <b>9</b>  |
| ZWIĄZKI 1:N (JEDEN DO WIELU) .....                                  | 9         |
| ZWIĄZKI 1:1 (OGRANICZONE PRZEZ UNIQUE) .....                        | 10        |
| ZACHOWANIA PRZY KASOWANIU REKORDÓW (ON DELETE) .....                | 10        |
| <b>4. INDEKSY .....</b>                                             | <b>11</b> |
| <b>5. REGUŁY BIZNESOWE I WIĘZY INTEGRALNOŚCI .....</b>              | <b>12</b> |
| <b>6. DANE TESTOWE .....</b>                                        | <b>13</b> |
| <b>7. WIDOKI .....</b>                                              | <b>14</b> |
| <b>8. ZAAWANSOWANE ZAPYTANIA SQL .....</b>                          | <b>15</b> |
| 8.1. Z1. KLIENCI POWYŻEJ ŚREDNIEJ .....                             | 15        |
| 8.2. Z2. POPULARNOŚĆ STACJI .....                                   | 15        |
| 8.3. Z3. OSTATNIE WYPOŻYCZENIE ROWERU .....                         | 15        |
| 8.4. Z4. RANKING KLIENTÓW WG PRZYCHODU .....                        | 15        |
| 8.5. Z5. ROWERY NIGDY NIEMWYPOŻYCZONE .....                         | 16        |
| 8.6. Z6. ŚREDNI CZAS WG TARYFY .....                                | 16        |
| <b>9. FUNKCJE, PROCEDURY I WYZWALACZE PL/PGSQL .....</b>            | <b>17</b> |
| 9.1. ZESTAWIENIE OBIEKTÓW .....                                     | 17        |
| 9.2. WYZWALACZE .....                                               | 17        |
| Trigger 1: <i>trg_dok_sync_status</i> .....                         | 17        |
| Trigger 2: <i>trg_blokuj_zablokowanego</i> .....                    | 17        |
| Trigger 3: <i>trg_status_roweru</i> .....                           | 17        |
| Trigger 4: <i>trg_blokuj_z_zaleglosciami</i> .....                  | 18        |
| 9.3. FUNKCJE I PROCEDURY .....                                      | 18        |
| Funkcja <i>oblicz_oplate(p_wypozyczenie)</i> .....                  | 18        |
| Procedura <i>wypożycz_rower(p_klient, p_rower, p_stacja)</i> .....  | 18        |
| Procedura <i>zwroc_rower(p_wypozyczenie, p_stacja_zwrotu)</i> ..... | 18        |
| <b>10. TRANSAKcje I POZIOMY IZOLACJI .....</b>                      | <b>19</b> |
| 10.1. MECHANIZMY .....                                              | 19        |
| 10.2. SCENARIUSZ „RACE CONDITION NA WYPOŻYCZENIU” .....             | 19        |
| 10.3. SCENARIUSZ „NON-REPEATABLE READ PRZY RAPORCIE” .....          | 19        |
| 10.4. SCENARIUSZ „WRITE SKEW (OSTATNI ROWER NA STACJI)” .....       | 19        |

|                                                            |           |
|------------------------------------------------------------|-----------|
| 10.5. WZORZEC „RETRY” .....                                | 20        |
| <b>11. BEZPIECZEŃSTWO DANYCH: ROLE I UPRAWNIENIA .....</b> | <b>21</b> |
| 11.1. ARCHITEKTURA RÓL .....                               | 21        |
| 11.2. POLITYKA „DENY BY DEFAULT” .....                     | 21        |
| 11.3. UPRAWNIENIA KOLUMNOWE .....                          | 21        |
| 11.4. ROW-LEVEL SECURITY (RLS) .....                       | 21        |
| 11.5. PROCEDURY JAKO BRAMKI BEZPIECZEŃSTWA .....           | 22        |
| 11.6. TESTY WERYFIKACYJNE .....                            | 22        |
| 11.7. AUDYT UPRAWNIENI .....                               | 22        |
| <b>12. INSTRUKCJA URUCHOMIENIA .....</b>                   | <b>23</b> |
| KROK 1. Utworzenie bazy .....                              | 23        |
| KROK 2. Uruchomienie skryptów .....                        | 23        |
| KROK 3. Weryfikacja .....                                  | 23        |
| KROK 4. Testy funkcjonalne .....                           | 23        |
| RESET BAZY .....                                           | 23        |
| <b>13. NORMALIZACJA, ZGODNOŚĆ Z 3NF .....</b>              | <b>24</b> |
| 1NF, Pierwsza Postać Normalna .....                        | 24        |
| 2NF, Druga Postać Normalna .....                           | 24        |
| 3NF, Trzecia Postać Normalna .....                         | 24        |
| <b>14. PODSUMOWANIE .....</b>                              | <b>25</b> |

## 1. Wprowadzenie i cel projektu

Projekt relacyjnej bazy danych obsługującej system wypożyczalni rowerów miejskich, obejmuje cykl od projektu logicznego, przez dane testowe, mechanizmy proceduralne i transakcyjne, po warstwę bezpieczeństwa.

System udostępnia:

- rejestrację klientów z przypisanymi taryfami cenowymi,
- zarządzanie flotą rowerów i ich statusem,
- obsługę stacji dokujących z dowolnym układem stanowisk,
- ewidencję wypożyczeń jedno- i wielostronnych (odbór i zwrot na różnych stacjach),
- rozliczenia finansowe oraz obsługę zaległości,
- zgłoszenia serwisowe i przypisanie rowerów do pracowników technicznych,
- atomowe operacje wypożyczenia i zwrotu z obsługą równoległości,
- kontrolę dostępu na poziomie ról funkcyjnych i wierszy (Row-Level Security).

**SZBD:** PostgreSQL (zgodny z SQL:2016). Schemat zachowuje 3NF (Trzecia Postać Normalna).

**Repozytorium:** <https://github.com/fish3er/db-bike-rental>

**Struktura plików projektu:**

| Plik                    | Zawartość                                |
|-------------------------|------------------------------------------|
| 01_ddl.sql              | Schemat (tabele, ograniczenia, indeksy)  |
| 02_dane.sql             | Dane testowe                             |
| 03_widoki_zapytania.sql | Widoki i zaawansowane zapytania          |
| 04_funkcje_triggery.sql | Funkcje, procedury i wyzwalacze PL/pgSQL |
| 05_transakcje.sql       | Transakcje i poziomy izolacji            |
| 06_bezpieczenstwo.sql   | Role, uprawnienia, Row-Level Security    |

## 2. Model danych, opis tabel

Baza składa się z 9 tabel połączonych kluczami obcymi. Kolejność tworzenia respektuje zależności FK: tabele referencyjne powstają przed tabelami zależnymi.

### 2.1. taryfa, cennik usług

Stawka stanowi osobną tabelę zamiast atrybutu klienta (eliminuje to problem aktualizacji każdego wiersza z daną taryfą i wprowadzenia zależności przechodniej klient → taryfa → stawka).

| Kolumna           | Typ          | Ograniczenie       | Domyślna | Opis                              |
|-------------------|--------------|--------------------|----------|-----------------------------------|
| id_taryfa         | SERIAL       | PRIMARY KEY        | auto     | Identyfikator taryfy              |
| nazwa             | VARCHAR(50)  | NOT NULL<br>UNIQUE | brak     | Unikalna nazwa cennika            |
| opłata_początkowa | NUMERIC(6,2) | >= 0               | 0        | Opłata startowa za wypożyczenie   |
| stawka_minuta     | NUMERIC(6,2) | >= 0               | brak     | Stawka za każdą rozpoczętą minutę |

### 2.2. klient, użytkownicy systemu

Każdy klient ma przypisany cennik (FK do taryfa), pole zablokowany wstrzymuje dostęp przy zaległościach lub naruszeniach, CHECK na emailu wymaga znaku @ nie na pierwszej pozycji.

| Kolumna     | Typ          | Ograniczenie                    | Domyślna | Opis                            |
|-------------|--------------|---------------------------------|----------|---------------------------------|
| id_klient   | SERIAL       | PRIMARY KEY                     | auto     | Identyfikator klienta           |
| email       | VARCHAR(120) | NOT NULL<br>UNIQUE,<br>CHECK(@) | brak     | Adres e-mail (unikalny)         |
| imie        | VARCHAR(50)  | NOT NULL                        | brak     | Imię klienta                    |
| nazwisko    | VARCHAR(50)  | NOT NULL                        | brak     | Nazwisko klienta                |
| id_taryfa   | INTEGER      | FK → taryfa                     | brak     | Przypisana taryfa               |
| zablokowany | BOOLEAN      | NOT NULL                        | FALSE    | Blokada możliwości wypożyczenia |

### 2.3. stacja, lokalizacje wypożyczalni

Stacja dokująca przechowuje współrzędne GPS i pojemność (liczbę stanowisk), CHECK na współrzędnych ogranicza je do prawidłowych zakresów (szerokość  $\pm 90^\circ$ , długość  $\pm 180^\circ$ ).

| Kolumna   | Typ          | Ograniczenie         | Domyślna | Opis                   |
|-----------|--------------|----------------------|----------|------------------------|
| id_stacja | SERIAL       | PRIMARY KEY          | auto     | Identyfikator stacji   |
| nazwa     | VARCHAR(100) | NOT NULL             | brak     | Nazwa lokalizacji      |
| pojemnosc | INTEGER      | > 0                  | brak     | Całkowita liczba doków |
| szer_geo  | NUMERIC(9,6) | BETWEEN -90 AND 90   | brak     | Szerokość geograficzna |
| dlug_geo  | NUMERIC(9,6) | BETWEEN -180 AND 180 | brak     | Długość geograficzna   |

### 2.4. rower, flota pojazdów

Status roweru przyjmuje jedną z trzech wartości (reguła RB1), przebieg jest nieujemny.

| Kolumna     | Typ         | Ograniczenie                             | Domyślna   | Opis                          |
|-------------|-------------|------------------------------------------|------------|-------------------------------|
| id_rower    | SERIAL      | PRIMARY KEY                              | auto       | Identyfikator roweru          |
| nr_seryjny  | VARCHAR(40) | NOT NULL UNIQUE                          | brak       | Numer seryjny (unikalny)      |
| status      | VARCHAR(20) | IN ('dostepny', 'wypozyczony', 'serwis') | 'dostepny' | Stan roweru (RB1)             |
| przebieg_km | INTEGER     | >= 0                                     | 0          | Łączny przebieg w kilometrach |

### 2.5. dok, stanowiska dokujące

Rozbicie pojemności stacji na poszczególne stanowiska, UNIQUE id\_rower gwarantuje, że rower nie stoi w dwóch dokach jednocześnie (RB2), CHECK utrzymuje spójność statusów: dok zajęty zawiera rower, dok wolny nie zawiera (RB3).

| Kolumna | Typ    | Ograniczenie | Domyślna | Opis                     |
|---------|--------|--------------|----------|--------------------------|
| id_dok  | SERIAL | PRIMARY KEY  | auto     | Identyfikator stanowiska |

| Kolumna   | Typ         | Ograniczenie          | Domyślna | Opis                                   |
|-----------|-------------|-----------------------|----------|----------------------------------------|
| id_stacja | INTEGER     | FK → stacja           | brak     | Stacja, do której należy dok           |
| id_rower  | INTEGER     | FK → rower,<br>UNIQUE | NULL     | Rower w doku<br>(NULL = pusty,<br>RB2) |
| status    | VARCHAR(20) | IN ('wolny','zajety') | 'wolny'  | Stan doku spójny z<br>id_rower (CHECK) |

## 2.6. wypożyczenie, historia wypożyczeń

Centralna tabela, pola start i koniec obsługują wypożyczenia jednostronne (odbior i zwrot na różnych stacjach), NULL w czas\_koniec oznacza wypożyczenie aktywne, czyli rower w trasie.

| Kolumna          | Typ       | Ograniczenie       | Domyślna | Opis                               |
|------------------|-----------|--------------------|----------|------------------------------------|
| id_wypozyczenie  | SERIAL    | PRIMARY KEY        | auto     | Identyfikator zdarzenia            |
| id_klient        | INTEGER   | FK → klient        | brak     | Klient wypożyczający               |
| id_rower         | INTEGER   | FK → rower         | brak     | Wypożyczony rower                  |
| id_stacja_start  | INTEGER   | FK → stacja        | brak     | Stacja odbioru                     |
| id_stacja_koniec | INTEGER   | FK → stacja, NULL  | NULL     | Stacja zwrotu<br>(NULL = w trasie) |
| czas_start       | TIMESTAMP | NOT NULL           | now()    | Moment podjęcia roweru             |
| czas_koniec      | TIMESTAMP | NULL >= czas_start | NULL     | Moment zwrotu<br>(NULL = aktywne)  |

## 2.7. płatność, rozliczenia finansowe

Płatność wiąże się z klientem i opcjonalnie z konkretną transakcją wypożyczenia, Status zaległa uruchamia regułę RB5.

| Kolumna     | Typ     | Ograniczenie | Domyślna | Opis                        |
|-------------|---------|--------------|----------|-----------------------------|
| id_platnosc | SERIAL  | PRIMARY KEY  | auto     | Identyfikator płatności     |
| id_klient   | INTEGER | FK → klient  | brak     | Klient obciążony płatnością |

| Kolumna         | Typ          | Ograniczenie              | Domyślna  | Opis                     |
|-----------------|--------------|---------------------------|-----------|--------------------------|
| id_wypozyczenie | INTEGER      | FK → wypozyczenie, NULL   | NULL      | Powiązane wypożyczenie   |
| kwota           | NUMERIC(8,2) | >= 0                      | brak      | Kwota do zapłaty         |
| status          | VARCHAR(20)  | IN ('oplacona','zalegla') | 'zalegla' | Status rozliczenia (RB5) |
| czas            | TIMESTAMP    | NOT NULL                  | now()     | Czas wystawienia         |

## 2.8. pracownik, personel serwisu

Pracownicy obsługujący zgłoszenia techniczne, pole rola opisuje stanowisko (technik, koordynator, administrator).

| Kolumna      | Typ         | Ograniczenie | Domyślna | Opis                                   |
|--------------|-------------|--------------|----------|----------------------------------------|
| id_pracownik | SERIAL      | PRIMARY KEY  | auto     | Identyfikator pracownika               |
| imie         | VARCHAR(50) | NOT NULL     | brak     | Imię                                   |
| nazwisko     | VARCHAR(50) | NOT NULL     | brak     | Nazwisko                               |
| rola         | VARCHAR(40) | NOT NULL     | brak     | Stanowisko (technik, koordynator itd.) |

## 2.9. serwis, zgłoszenia techniczne

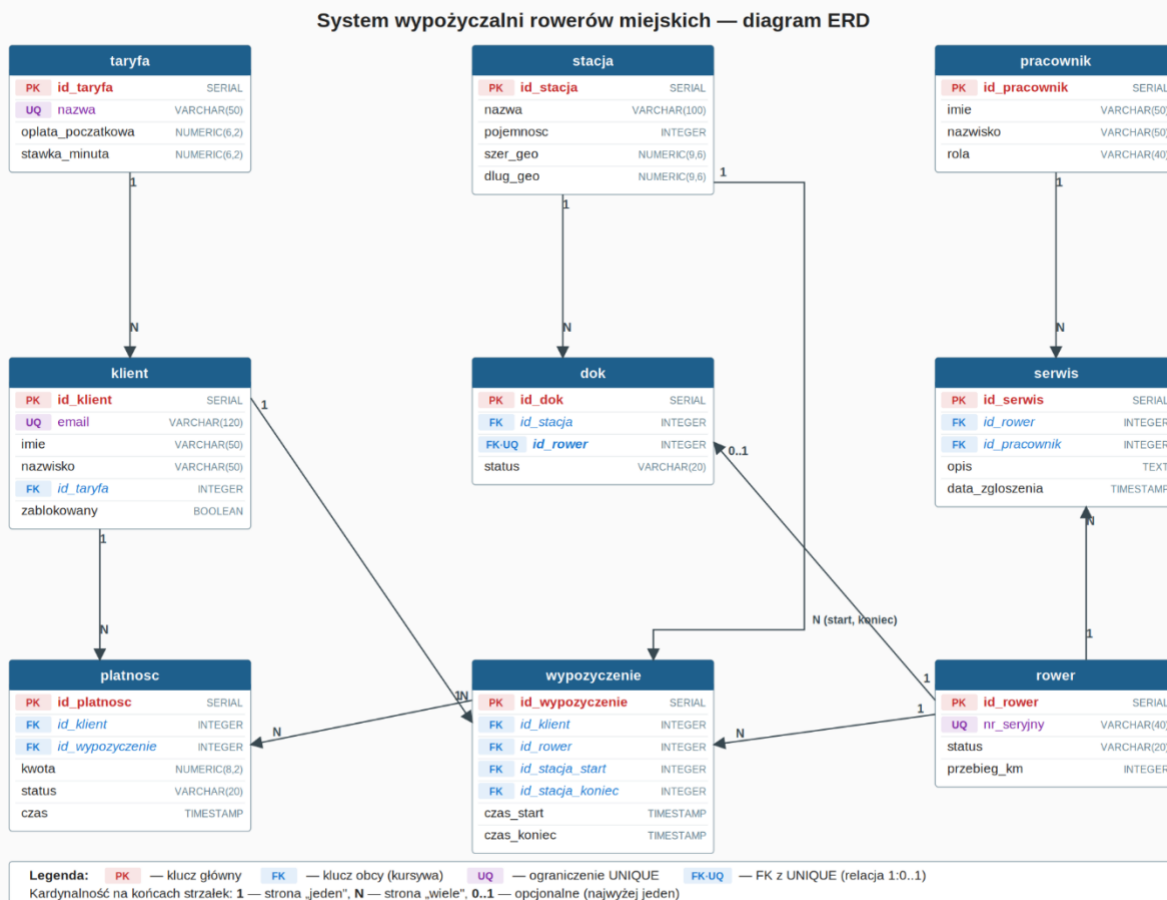
Każde zgłoszenie dotyczy jednego roweru i opcjonalnie pracownika, NULL w id\_pracownik oznacza zgłoszenie bez przypisanego technika.

| Kolumna         | Typ       | Ograniczenie         | Domyślna | Opis                                      |
|-----------------|-----------|----------------------|----------|-------------------------------------------|
| id_serwis       | SERIAL    | PRIMARY KEY          | auto     | Identyfikator zgłoszenia                  |
| id_rower        | INTEGER   | FK → rower           | brak     | Rower wymagający naprawy                  |
| id_pracownik    | INTEGER   | FK → pracownik, NULL | NULL     | Przypisany technik (NULL = nieprzypisane) |
| opis            | TEXT      | NOT NULL             | brak     | Treść zgłoszenia                          |
| data_zgloszenia | TIMESTAMP | NOT NULL             | now()    | Data i godzina zgłoszenia                 |



### 3. Diagram związków encji (ERD)

Diagram przedstawia 9 tabel oraz 11 powiązań kluczy obcych. Kolorystyka: nagłówki niebieskie, klucze główne (PK) czerwone, klucze obce (FK) niebieskie kursywą, ograniczenia UNIQUE fioletowe. Etykiety 1/N przy końcach strzałek oznaczają kardynalność relacji.



**Rys. 1.** Diagram ERD systemu wypożyczalni rowerów miejskich. 9 tabel, 11 powiązań kluczy obcych, schemat zgodny z 3NF.

#### Związki 1:N (jeden do wielu)

- taryfa → klient: taryfa przypisana do wielu klientów
- klient → wypozyczenie: klient ma historię wypożyczeń
- rower → wypozyczenie: rower ma historię wypożyczeń
- stacja → wypozyczenie (start i koniec): stacja przyjmuje wiele wypożyczeń
- stacja → dok: stacja dzieli się na wiele doków
- klient → platnosc: klient ma historię płatności
- wypozyczenie → platnosc: wypożyczenie może generować wiele płatności
- rower → serwis: rower ma historię zgłoszeń serwisowych
- pracownik → serwis: pracownik obsługuje wiele zgłoszeń

## Związki 1:1 (ograniczone przez UNIQUE)

- `dok.id_rower` UNIQUE: rower zadokowany w co najwyżej jednym stanowisku (RB2)

## Zachowania przy kasowaniu rekordów (ON DELETE)

| Relacja                 | Zachowanie                                                   |
|-------------------------|--------------------------------------------------------------|
| taryfa → klient         | RESTRICT (taryfa używana przez klienta nie zostaje usunięta) |
| stacja → dok            | CASCADE (usunięcie stacji usuwa jej doki)                    |
| rower → dok             | SET NULL (usunięcie roweru zwalnia dok)                      |
| pracownik → serwis      | SET NULL (zgłoszenie zostaje bez przypisania)                |
| wypożyczenie → platnosc | SET NULL (płatność zostaje bez powiązania)                   |
| rower → serwis          | CASCADE (usunięcie roweru usuwa jego zgłoszenia)             |
| klient → wypożyczenie   | RESTRICT (klient z historią wypożyczeń pozostaje)            |

## 4. Indeksy

Oprócz indeksów na PRIMARY KEY i UNIQUE skrypt DDL tworzy sześć dodatkowych indeksów wspierających najczęstsze zapytania.

| Nazwa indeksu        | Tabela / kolumna                                    | Cel                                    |
|----------------------|-----------------------------------------------------|----------------------------------------|
| idx_wyp_klient       | wypozyczenie(id_klient)                             | Historia wypożyczeń klienta            |
| idx_wyp_rower        | wypozyczenie(id_rower)                              | Wypożyczenia danego roweru             |
| idx_wyp_aktywne      | wypozyczenie(id_rower) WHERE<br>czas_koniec IS NULL | Indeks częściowy, aktywne wypożyczenia |
| idx_dok_stacja       | dok(id_stacja)                                      | Zliczanie doków na stacji              |
| idx_platnosc_zalegla | platnosc(id_klient, status)                         | Wykrywanie zaległości (RB5)            |
| idx_serwis_rower     | serwis(id_rower)                                    | Zgłoszenia serwisowe roweru            |

## 5. Reguły biznesowe i więzy integralności

Każdą regułę realizuje warstwa deklaratywna (CHECK, UNIQUE, FK), warstwa proceduralna (trigger w pliku 04), lub jedna i druga jednocześnie.

| ID  | Reguła                                  | Implementacja w bazie                                                                                                      |
|-----|-----------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| RB1 | Rower ma jeden z trzech stanów          | CHECK status IN ('dostępny', 'wypożyczony', 'serwis') plus trigger trg_status_roweru                                       |
| RB2 | Rower zadokowany w jednym miejscu       | UNIQUE(id_rower) na tabeli dok                                                                                             |
| RB3 | Dok zajęty zawiera rower, dok wolny nie | CHECK (status='zajety' AND id_rower IS NOT NULL) OR (status='wolny' AND id_rower IS NULL) plus trigger trg_dok_sync_status |
| RB4 | Klient zablokowany nie może wypożyczać  | Trigger trg_blokuj_zablokowanego (RAISE EXCEPTION)                                                                         |
| RB5 | Zaległa płatność blokuje klienta        | Trigger trg_blokuj_z_zaleglosciami plus indeks idx_platnosc_zalegla plus widok rozliczenia_klientow                        |
| RB6 | Czas zakończenia >= czas rozpoczęcia    | CHECK (czas_koniec IS NULL OR czas_koniec >= czas_start)                                                                   |
| RB7 | Email klienta zawiera znak @            | CHECK (POSITION('@' IN email) > 1)                                                                                         |

## 6. Dane testowe

**Plik:** 02\_dane.sql

Skrypt wypełnia bazę zestawem danych, w którym widoki i zapytania zwracają zróżnicowane wyniki: różną liczbę wypożyczeń na klienta, opłacone i zaległe płatności, aktywne i zakończone sesje.

| Kategoria            | Zawartość                                                                                    |
|----------------------|----------------------------------------------------------------------------------------------|
| Taryfy               | 3 taryfy: Standard (2,00 + 0,50/min), Student (1,00 + 0,30/min), Abonament (0,00 + 0,20/min) |
| Klienci              | 8 klientów (1 zablokowany: Zofia Lewandowska, 1 z zaległością: Tomasz Zieliński)             |
| Stacje               | 4 stacje: Centrum Metro, Mokotów Pole Mokotowskie, Wola Rondo Daszyńskiego, Praga ZOO        |
| Doki                 | 26 stanowisk dokujących (8+6+6+6), z różnym stopniem zapelnienia                             |
| Rowery               | 12 rowerów: 7 dostępnych w dokach, 2 wypożyczone (aktywne), 3 w serwisie                     |
| Pracownicy           | 4 pracowników: 2 techników, 1 koordynator, 1 administrator                                   |
| Wypożyczenia         | 20 rekordów: 18 zakończonych plus 2 aktywne (rower 8 i 9)                                    |
| Płatności            | 18 rekordów: 17 opłaconych plus 1 zaległa (Tomasz Zieliński, wypożyczenie 18)                |
| Zgłoszenia serwisowe | 3 zgłoszenia dla rowerów RW-0010, RW-0011, RW-0012                                           |

## 7. Widoki

**Plik:** 03\_widoki\_zapytania.sql, część A

Cztery widoki pokrywają potrzeby operacyjne i analityczne systemu.

| Widok                | Opis                                                                                                                              |
|----------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| dostepnosc_stacji    | Liczba dostępnych rowerów, wolnych doków i procent zapelnienia każdej stacji. Widok operacyjny do podglądu w czasie rzeczywistym. |
| historia_wypozycczen | Zestawienie wypożyczeń z danymi klienta, roweru, stacji startu i zwrotu oraz wyliczonym czasem trwania w minutach.                |
| rozliczenia_klientow | Podsumowanie finansowe per klient: liczba wypożyczeń, suma opłacona i suma zaległa.                                               |
| rowery_do_serwisu    | Rowery o statusie serwis wraz z najnowszym zgłoszeniem i przypisanym technikiem (lub etykietą „nieprzypisane”).                   |

## 8. Zaawansowane zapytania SQL

Plik: 03\_widoki\_zapytania.sql, część B

| Nr | Technika SQL                  | Wynik                                                |
|----|-------------------------------|------------------------------------------------------|
| Z1 | Podzapytanie w HAVING         | Klienci z liczbą wypożyczeń powyżej średniej         |
| Z2 | GROUP BY plus HAVING          | Popularność stacji startowych                        |
| Z3 | Podzapytanie skorelowane      | Najnowsze wypożyczenie każdego roweru                |
| Z4 | Funkcja okna (RANK, SUM OVER) | Ranking klientów wg przychodu z udziałem procentowym |
| Z5 | NOT EXISTS                    | Rowery bez żadnego wypożyczenia                      |
| Z6 | CTE plus agregacja            | Średni czas wypożyczenia w rozbiciu na taryfę        |

### 8.1. Z1. Klienci powyżej średniej

```
SELECT k.imie || ' ' || k.nazwisko AS klient, count(*) AS wypozycczen
FROM wypozycczenie w JOIN klient k ON k.id_klient = w.id_klient
GROUP BY k.id_klient, k.imie, k.nazwisko
HAVING count(*) > (
    SELECT count(*)::numeric / count(DISTINCT id_klient) FROM wypozycczenie
);
```

### 8.2. Z2. Popularność stacji

```
SELECT s.nazwa AS stacja, count(*) AS liczba_startow
FROM wypozycczenie w JOIN stacja s ON s.id_stacja = w.id_stacja_start
GROUP BY s.id_stacja, s.nazwa
HAVING count(*) >= 1
ORDER BY liczba_startow DESC;
```

### 8.3. Z3. Ostatnie wypożyczenie roweru

```
SELECT r.nr_seryjny, w.czas_start, w.id_klient
FROM wypozycczenie w JOIN rower r ON r.id_rower = w.id_rower
WHERE w.czas_start = (
    SELECT max(w2.czas_start)
    FROM wypozycczenie w2
    WHERE w2.id_rower = w.id_rower -- korelacja
);
```

### 8.4. Z4. Ranking klientów wg przychodu

```
SELECT
    k.imie || ' ' || k.nazwisko AS klient,
    sum(p.kwota) AS przychod,
    RANK() OVER (ORDER BY sum(p.kwota) DESC) AS pozycja,
    round(100.0 * sum(p.kwota) / sum(sum(p.kwota)) OVER (), 1) AS udzial_proc
```

```
FROM platnosc p JOIN klient k ON k.id_klient = p.id_klient
WHERE p.status = 'oplacona'
GROUP BY k.id_klient, k.imie, k.nazwisko
ORDER BY przychod DESC;
```

## 8.5. Z5. Rowery nigdy niewypożyczone

```
SELECT r.nr_seryjny, r.status
FROM rower r
WHERE NOT EXISTS (
    SELECT 1 FROM wypozyczenie w WHERE w.id_rower = r.id_rower
);
```

## 8.6. Z6. Średni czas wg taryfy

```
WITH zakonczone AS (
    SELECT id_klient,
           EXTRACT(EPOCH FROM (czas_koniec - czas_start)) / 60.0 AS minuty
    FROM wypozyczenie WHERE czas_koniec IS NOT NULL
)
SELECT t.nazwa AS taryfa, count(*) AS liczba,
       round(avg(z.minuty)::numeric, 1) AS sredni_czas_min
FROM zakonczone z
JOIN klient k ON k.id_klient = z.id_klient
JOIN taryfa t ON t.id_taryfa = k.id_taryfa
GROUP BY t.nazwa ORDER BY sredni_czas_min DESC;
```



## 9. Funkcje, procedury i wyzwalacze PL/pgSQL

**Plik:** 04\_funkcje\_triggery.sql

Dodanie warstwy proceduralnej. Uzupełnienie triggerami deklaracyjnych ograniczeń schematu zależności między tabelami, kaskadowej zmiany stanu.

### 9.1. zestawienie obiektów

| Obiekt                                                    | Typ            | Reguła / cel                                                              |
|-----------------------------------------------------------|----------------|---------------------------------------------------------------------------|
| fn_dok_sync_status /<br>trg_dok_sync_status               | trigger BEFORE | RB3, auto-sync dok.status z dok.id_rower                                  |
| fn_blokuj_zablokowanego /<br>trg_blokuj_zablokowanego     | trigger BEFORE | RB4, RAISE EXCEPTION przy próbie wypożyczenia przez zablokowanego klienta |
| fn_status_roweru /<br>trg_status_roweru                   | trigger AFTER  | RB1, synchronizacja rower.status ze stanem wypożyczenia                   |
| fn_blokuj_z_zaleglosciami /<br>trg_blokuj_z_zaleglosciami | trigger AFTER  | RB5, blokada klienta przy płatności zaległa                               |
| oblicz_oplate(p_wypozyczenie)                             | funkcja STABLE | Kwota wg taryfy:<br>opłata_poczatkowa + CEIL(minuty)<br>* stawka_minuta   |
| wypożycz_rower(p_klient,<br>p_rower, p_stacja)            | procedura      | Atomowe rozpoczęcie wypożyczenia z SELECT FOR UPDATE                      |
| zwroc_rower(p_wypozyczenie,<br>p_stacja_zwrotu)           | procedura      | Atomowy zwrot, zadokowanie i naliczenie opłaty                            |

### 9.2. wyzwalacze

#### Trigger 1: trg\_dok\_sync\_status

CHECK ze schematu wykrywa naruszenie RB3, ale wymaga, by aplikacja sama ustawiała oba pola spójnie. Trigger przejmie tę odpowiedzialność: po zmianie dok.id\_rower ustawia dok.status (NULL → 'wolny', rower → 'zajety'). Aplikacja modyfikuje pojedyncze pole.

#### Trigger 2: trg\_blokuj\_zablokowanego

Realizuje RB4 ostatecznie: blokuje INSERT do wypozyczenie dla klienta z flagą zablokowany = TRUE. Rzuca błąd z kodem SQLSTATE check\_violation. Ochrona działa niezależnie od warstwy aplikacji.

#### Trigger 3: trg\_status\_roweru

Utrzymuje rower.status spójny z wypożyczeniami:

- INSERT wypożyczenia: status := 'wypożyczony'
- UPDATE z czas\_koniec IS NOT NULL: status := 'dostępny'

Status serwis ma pierwszeństwo i nie zostaje nadpisany. Rower w serwisie nie powinien mieć aktywnego wypożyczenia, a gdyby się tak zdarzyło, status serwisowy się utrzymuje.

#### Trigger 4: trg\_blokuj\_z\_zaleglosciami

Reaguje na INSERT, UPDATE i DELETE w platnosc. Synchronizuje klient.zablokowany z istnieniem płatności zaległa. Klient z zaległą płatnością dostaje flagę TRUE; klient bez zaległości dostaje FALSE. Warunek IS DISTINCT FROM eliminuje puste zapisy.

### 9.3. funkcje i procedury

#### Funkcja oblicz\_oplate(p\_wypożyczenie)

Oznaczenie STABLE informuje optymalizator, że funkcja czyta dane, ale nie modyfikuje stanu bazy. Dla wypożyczeń aktywnych (czas\_koniec IS NULL) liczy opłatę na bieżącą chwilę przez now(). Formuła: opłata\_początkowa + CEIL(minuty) \* stawka\_minuta.

#### Procedura wypożycz\_rower(p\_klient, p\_rower, p\_stacja)

1. SELECT status FROM rower WHERE id\_rower = p\_rower FOR UPDATE, blokada wiersza na czas transakcji
2. Sprawdzenie status = 'dostępny' (wyjątek przy innej wartości)
3. INSERT do wypożyczenie, uruchamiający trigger RB4 i RB1
4. UPDATE doku (id\_rower := NULL), uruchamiający trg\_dok\_sync\_status FOR UPDATE blokuje dwie sesje rezerwujące ten sam rower

#### Procedura zwroc\_rower(p\_wypożyczenie, p\_stacja\_zwrotu)

1. SELECT FOR UPDATE na wypożyczeniu, blokada przed równoległym zwrotem
2. UPDATE wypożyczenia (czas\_koniec, id\_stacja\_koniec), trigger przywracający rower.status
3. Znalezienie wolnego doku (SELECT FOR UPDATE LIMIT 1) i zadokowanie
4. Przy braku wolnego doku: RAISE WARNING (wypożyczenie i tak się zamyka)
5. Naliczenie opłaty przez oblicz\_oplate(), INSERT do platnosc ze statusem zaległa, trigger RB5 blokuje klienta do czasu zapłaty

## 10. Transakcje i poziomy izolacji

Plik: 05\_transakcje.sql

Mechanizmy transakcyjne PostgreSQL na domenę wypożyczalni. Scenariusze 10.2–10.4 demonstrują anomalie przy zbyt słabym poziomie izolacji oraz jej rozwiązanie.

### 10.1. mechanizmy

- jawne sterowanie transakcją: BEGIN, COMMIT, ROLLBACK
- punkty zachowawcze: SAVEPOINT, ROLLBACK TO SAVEPOINT
- bloki EXCEPTION w PL/pgSQL (niejawny savepoint pod spodem)
- trzy poziomy izolacji: READ COMMITTED, REPEATABLE READ, SERIALIZABLE
- blokady pesymistyczne: SELECT ... FOR UPDATE
- wzorzec retry przy serialization\_failure

### 10.2. scenariusz „race condition na wypożyczeniu”

Bez blokady dwie sesje wypożyczają ten sam rower:

```

SESJA A: SELECT status FROM rower WHERE id_rower=1; -- 'dostępny'
SESJA B: SELECT status FROM rower WHERE id_rower=1; -- 'dostępny'
SESJA A: INSERT INTO wypozyczenie ...; COMMIT;
SESJA B: INSERT INTO wypozyczenie ...; COMMIT;      -- też się udało

```

**Rozwiązanie:** SELECT ... FOR UPDATE w procedurze wypozyucz\_rower() z pliku 04. Druga sesja czeka do COMMIT pierwszej, po czym odczytuje już status = 'wypożyczony' i otrzymuje wyjątek.

### 10.3. scenariusz „non-repeatable read przy raporcie”

Koordinator generuje raport „suma opłaconych plus suma zaległych” jako dwa kolejne SELECT. W trakcie raportu inna sesja opłaca jedną z zaległych płatności. Przy READ COMMITTED ta sama kwota wpada do obu sum, raport się nie zgadza.

**Rozwiązanie:** BEGIN ISOLATION LEVEL REPEATABLE READ. PostgreSQL ustala snapshot przy pierwszym poleceniu transakcji i utrzymuje go do COMMIT/ROLLBACK. Kolejne SELECT widzą ten sam stan niezależnie od zmian w innych sesjach.

### 10.4. scenariusz „write skew (ostatni rower na stacji)”

Polityka biznesowa nie pozwala wypożyczyć ostatniego roweru na stacji (rezerwa dla obsługi). Dwie sesje sprawdzają liczbę dostępnych rowerów. Każda widzi „2” i decyduje, że może wypożyczyć (zostanie 1). Po COMMIT obu sesji zostaje 0, polityka złamana.

**Rozwiązanie:** BEGIN ISOLATION LEVEL SERIALIZABLE. PostgreSQL używa Serializable Snapshot Isolation (SSI). Silnik śledzi zależności read/write między równoległymi transakcjami i

przy COMMIT odrzuca jedną z konfliktujących transakcji błędem SQLSTATE 40001 (serialization\_failure). Aplikacja obsługuje retry.

## 10.5. wzorzec „retry”

Implementacja dla serialization\_failure:

```
LOOP
  BEGIN
    CALL wypożycz_rower(...);
    EXIT;
  EXCEPTION WHEN serialization_failure THEN
    IF v_proba >= p_max_prob THEN RAISE EXCEPTION ...;
    END IF;
    PERFORM pg_sleep(0.05 * v_proba); -- backoff
  END;
END LOOP;
```

## 11. Bezpieczeństwo danych: role i uprawnienia

Plik: 06\_bezpieczenstwo.sql

Warstwa kontroli dostępu zgodną z zasadą najmniejszego uprawnienia (least privilege).

### 11.1. architektura ról

Pięć ról grupowych (NOLOGIN) modeluje konteksty funkcyjne. Trzy konta użytkowe (LOGIN) dziedziczą uprawnienia z grup.

| Rola grupowa    | Przeznaczenie          | Kluczowe uprawnienia                                                                   |
|-----------------|------------------------|----------------------------------------------------------------------------------------|
| bike_admin      | Administrator schematu | ALL na wszystkim                                                                       |
| bike_operator   | Obsługa wypożyczeń     | SELECT/INSERT/UPDATE na wypożyczenie, platnosc, dok; SELECT na rower i stacja          |
| bike_serwisant  | Serwis floty           | SELECT/INSERT/UPDATE na serwis; GRANT UPDATE (status) ON rower (uprawnienie kolumnowe) |
| bike_klient_api | Aplikacja kliencka     | SELECT/UPDATE na własnych danych przez RLS; INSERT na wypożyczenie                     |
| bike_raporty    | Read-only BI           | SELECT na wszystkich tabelach                                                          |

Konta demonstracyjne: anna\_operator (operator), jan\_serwisant (serwisant), klient\_demo (klient API); hasła przykładowe; środowisko produkcyjne wymaga silnego uwierzytelniania (SCRAM lub cert).

### 11.2. polityka „deny by default”

REVOKE ALL ON ... FROM PUBLIC cofa domyślne uprawnienia, które PostgreSQL nadaje wszystkim użytkownikom, nadaje per rola to, co konieczne; ALTER DEFAULT PRIVILEGES zabezpiecza przyszłe obiekty.

### 11.3. uprawnienia kolumnowe

Serwisant modyfikuje tylko kolumnę status w tabeli rower, bez dostępu do przebieg\_km ani nr\_seryjny:

```
GRANT SELECT          ON rower TO bike_serwisant;
GRANT UPDATE (status) ON rower TO bike_serwisant;
```

### 11.4. Row-Level Security (RLS)

Logowanie klienta bez API zwraca wyłącznie własne rekordy w klient i wypożyczenie, nawet przy SELECT \* bez warunku WHERE.

```
ALTER TABLE klient ENABLE ROW LEVEL SECURITY;

CREATE POLICY pol_klient_wlasne_dane ON klient
  FOR ALL TO bike_klient_api
  USING (id_klient = COALESCE(
    NULLIF(current_setting('app.current_klient', true), ''),
    '0')::INTEGER)
  WITH CHECK (id_klient = COALESCE(
    NULLIF(current_setting('app.current_klient', true), ''),
    '0')::INTEGER);
```

Aplikacja po uwierzytelnieniu wykonuje `SET LOCAL app.current_klient = '<id_klient>';` polityka RLS porównuje `id_klient` z tym ustawieniem; `SET LOCAL` usuwa ustawienie z końcem transakcji, zapobiega wyciekowi tożsamości między żądaniami; klauzula `WITH CHECK` zabezpiecza przed zmianą cudzych danych.

### 11.5. procedury jako bramki bezpieczeństwa

Uprawnienia `EXECUTE` są nadawane wybiórczo:

- `bike_operator`: `wypozycz_rower`, `zwroc_rower`, `oblicz_oplate`
- `bike_klient_api`: `wypozycz_rower`, `oblicz_oplate` (bez `zwroc_rower`, zwrot wykonuje pracownik stacji)
- `bike_raporty`: tylko `oblicz_oplate` (do raportów)

Procedury pełnią rolę API bazodanowego, aplikacja kliencka wywołuje wcześniej zatwierdzone procedury.

### 11.6. testy weryfikacyjne

Pięć bloków testowych wykorzystujących `SET ROLE`. `SET ROLE` przełącza efektywną tożsamość bez nowego logowania, każdy test sprawdza, czy odrzucenie operacji nastąpiło z właściwej przyczyny; bloki `EXCEPTION WHEN insufficient_privilege` do potwierdzenia, że błąd dotyczył uprawnień.

Test końcowy demonstruje działanie RLS: `SET ROLE bike_klient_api` i `SET LOCAL app.current_klient = '1'`.

### 11.7. audyt uprawnień

Sekcja H zawiera zapytania pomocnicze:

- lista ról systemowych i ich przynależności (`pg_roles`, `pg_auth_members`),
- macierz uprawnień tabelowych (`information_schema.role_table_grants`),
- lista aktywnych polityk RLS (`pg_policies`).

## 12. Instrukcja uruchomienia

**Wymagania wstępne:** PostgreSQL 12 lub wyższy; właściciel bazy lub superuser dla skryptu 06

### Krok 1. utworzenie bazy

```
createdb bike_rental
```

### Krok 2. uruchomienie skryptów

Kolejność obligatoryjna:

```
psql -d bike_rental -f 01_ddl.sql           # schemat
psql -d bike_rental -f 02_dane.sql          # dane testowe
psql -d bike_rental -f 04_funkcje_triggery.sql # PL/pgSQL przed widokami
psql -d bike_rental -f 03_widoki_zapytania.sql # widoki i zapytania
psql -d bike_rental -f 05_transakcje.sql     # demo transakcji
psql -d bike_rental -f 06_bezpieczenstwo.sql # role i uprawnienia
```

### Krok 3. weryfikacja

```
psql -d bike_rental -c "\dt"                # lista tabel
psql -d bike_rental -c "\df"                # lista funkcji
psql -d bike_rental -c "\du"                # lista ról
psql -d bike_rental -c "SELECT * FROM dostepnosc_stacji;"
psql -d bike_rental -c "SELECT * FROM rozliczenia_klientow;"
```

### Krok 4. testy funkcjonalne

```
-- Test triggera RB4 (klient zablokowany):
CALL wypożycz_rower(8, 3, 1); -- powinno rzucić EXCEPTION

-- Test atomowego wypożyczenia:
CALL wypożycz_rower(1, 1, 1);
SELECT * FROM historia_wypożyczen ORDER BY id_wypożyczenie DESC LIMIT 1;

-- Test bezpieczeństwa kolumnowego:
SET ROLE bike_serwisant;
UPDATE rower SET przebieg_km = 0 WHERE id_rower = 1; -- ERROR
UPDATE rower SET status = 'serwis' WHERE id_rower = 1; -- OK
RESET ROLE;
```

### Reset bazy

Skrypty zawierają sekcje DROP TABLE/VIEW/FUNCTION/POLICY IF EXISTS CASCADE. Ponowne uruchomienie na istniejącym schemacie nie powoduje błędów.

## 13. Normalizacja, zgodność z 3NF

---

Schemat zachowuje Trzecią Postać Normalną (3NF).

### 1NF, Pierwsza Postać Normalna

- Każda komórka zawiera wartość atomową, bez list ani struktur zagnieżdżonych.
- Każda tabela ma klucz główny (SERIAL PRIMARY KEY).
- Wiersze są unikalne.

### 2NF, Druga Postać Normalna

- Atrybuty niekluczowe zależą funkcyjnie od całego klucza głównego.
- Klucze główne są jednokolumnowe (SERIAL), więc częściowe zależności nie występują.

### 3NF, Trzecia Postać Normalna

- Brak zależności przechodnich.
- Przykład: klient.id\_taryfa → taryfa.id\_taryfa → stawka\_minuta. Stawka leży w taryfa, nie w klient. Gdyby klient zawierał stawka\_minuta, zmiana cennika wymagałaby aktualizacji wszystkich klientów z daną taryfą, co tworzyłoby klasyczną anomalię aktualizacji.



## 14. Podsumowanie

Projekt obejmuje system bazy danych wypożyczalni rowerów miejskich na pełnym cyklu życia: od analizy wymagań i projektu logicznego, przez implementację z mechanizmami proceduralnymi, transakcyjnymi i bezpieczeństwa, po testowanie i dokumentację techniczną.

### Statystyki implementacji:

| Element                      | Liczba                       | Plik    |
|------------------------------|------------------------------|---------|
| Tabele                       | 9                            | 01      |
| Reguły biznesowe             | 7 (RB1–RB7)                  | 01 + 04 |
| Indeksy (w tym 1 częściowy)  | 6                            | 01      |
| Rekordy testowe              | ~95                          | 02      |
| Widoki                       | 4                            | 03      |
| Zaawansowane zapytania SQL   | 6                            | 03      |
| Wyzwalacze (triggery)        | 4                            | 04      |
| Funkcje i procedury PL/pgSQL | 4                            | 04 + 05 |
| Scenariusze transakcyjne     | 5                            | 05      |
| Role bezpieczeństwa          | 5 grupowych + 3 użytkowników | 06      |
| Polityki Row-Level Security  | 2                            | 06      |
| Testy weryfikacyjne          | 5 bloków                     | 06      |

Kod źródłowy: <https://github.com/fish3er/db-bike-rental>